Aluno: José Mário de Melo Brandão Guimarães

Parte 1 — Análise inicial de qualidade

1. O problema na fala do gerente é que ele está deixando os testes apenas para o final do desenvolvimento. Isso pode ser um problema porque alguns erros podem começar ainda nos requisitos ou nas regras de negócio.

2. O QA deveria participar desde o começo para ajudar a analisar os requisitos, encontrar informações que não estão claras e pensar nos possíveis cenários de teste.

3. Um dos riscos é encontrar um defeito grave quando o sistema já estiver próximo da entrega, causando atrasos. Outro risco é que um problema em uma funcionalidade pode afetar várias outras partes do sistema, fazendo com que a equipe precise alterar muita coisa para conseguir corrigir.

Parte 2 — Verificação dos requisitos

RQ-04 - diz que o paciente deve conseguir agendar uma consulta facilmente. O problema é a palavra “facilmente”, pois isso pode variar de uma pessoa para outra. Dessa forma, fica difícil testar esse requisito porque não existe uma regra dizendo o que deve ser considerado fácil. **Uma versão melhor seria:** “Pelo menos 90% dos usuários que participarem do teste devem conseguir agendar uma consulta em até dois minutos e sem precisar de ajuda”.

RQ-05 - diz que o paciente pode cancelar uma consulta quando necessário. O problema é que ele não explica quando o cancelamento pode ser realizado. Por exemplo, não sabemos se o paciente pode cancelar a qualquer momento ou se existe um prazo mínimo antes da consulta. Uma versão melhor seria: “O paciente autenticado poderá cancelar uma consulta até 24 horas antes do horário marcado. Quando a consulta for cancelada, o horário deverá voltar a ficar disponível para outros pacientes”.

RQ-07 - diz que o sistema deve ser rápido e seguro. O problema é que as palavras “rápido’ e “seguro” são muito vagas. O requisito não informa qual deve ser o tempo de resposta nem quais medidas de segurança devem ser usadas. Além disso, ele coloca desempenho e segurança dentro do mesmo requisito. **Uma versão melhor seria:** - O sistema deverá mostrar os horários disponíveis em até três segundos, mesmo com até 100 usuários acessando ao mesmo tempo. As senhas dos pacientes

deverão ser armazenadas utilizando hash, e a comunicação com o sistema deverá utilizar HTTPS.

Parte 3 — Verificação ou validação?

1. **VERIFICAÇÃO** - Revisar se o requisito de cancelamento está claro é uma atividade de **verificação**, pois estamos analisando um requisito antes de testar o sistema funcionando.

2. **VALIDAÇÃO** - Executar o login utilizando um e-mail e uma senha válidos é uma atividade de **validação**, pois estamos utilizando o sistema para conferir se ele funciona como deveria.

3. **VERIFICAÇÃO** - Conferir se o requisito de agendamento possui uma regra para horários disponíveis é uma atividade de **verificação**, pois estamos analisando se o requisito foi escrito de maneira completa.

4. **VALIDAÇÃO** - Testar se o sistema envia o e-mail depois do agendamento é uma atividade de **validação**, pois estamos verificando o funcionamento real dessa funcionalidade.

5. **VERIFICAÇÃO** - Analisar se o plano de testes cobre o cadastro, o login e o agendamento é uma atividade de **verificação**, já que estamos revisando um documento de planejamento.

6. **VALIDAÇÃO** - Simular um paciente cancelando uma consulta pelo sistema é uma atividade de **validação**, pois estamos testando como o sistema se comporta durante o seu uso.

Parte 4 — Erro, defeito ou falha?

1. **DEFEITO** - Quando o analista escreveu “cancelar quando necessário”, mas não definiu um prazo mínimo, temos um **defeito no requisito**. Nesse caso, faltou uma informação importante para que a equipe soubesse como o cancelamento deveria funcionar.

2. **DEFEITO** - Quando o código permite que uma consulta seja cancelada cinco minutos antes, mesmo que a regra correta seja de 24 horas, temos um **defeito**. Isso acontece porque a regra foi implementada de maneira incorreta no código.

3. **FALHA** - Quando o paciente cancela a consulta cinco minutos antes e o sistema aceita, temos uma **falha**. Aqui o defeito que existia no código apareceu durante a utilização do sistema.

4. **ERRO** - Quando o desenvolvedor entende de forma errada a regra de envio do e-mail, temos um **erro**, pois foi um engano cometido por uma pessoa.

5. **DEFEITO** - Quando a função responsável pelo envio do e-mail não é chamada depois do agendamento, temos um **defeito**, pois existe um problema dentro do código.

6. **FALHA** - Quando o paciente agenda uma consulta, mas não recebe o e-mail de confirmação, temos uma **falha**, pois o sistema apresentou um comportamento diferente do esperado durante o uso.

Parte 5 — Aplicando princípios de teste

1. Não seria possível testar todas as combinações porque existem muitas possibilidades diferentes dentro do sistema. Teríamos que testar vários tipos de e-mail, senha, pacientes, médicos, datas, horários, agendamentos e cancelamentos. Quando juntamos todas essas possibilidades, a quantidade de testes fica muito grande.

2. A funcionalidade que deveria receber mais atenção é o agendamento, pois ela é a principal função do sistema. Um defeito nessa parte poderia permitir que dois pacientes agendassem o mesmo horário ou que uma consulta fosse marcada em um horário que não está disponível.

3. Não encontrar nenhum bug não significa que o sistema está totalmente correto. Isso acontece porque os testes realizados podem não ter passado por todos os cenários possíveis.

4. Caso a equipe sempre teste os mesmos cenários, ela pode deixar de encontrar defeitos que aparecem em situações diferentes.